

Reporte de Práctica

Inclinómetro con Control de Motorreductor

Arduino UNO R4 WiFi + MPU-6050 + Puente H L298N

Objetivo

Implementar un sistema de control de inclinación que lea el ángulo de un sensor acelerómetro/giroscopio MPU-6050 y, en función de dicho ángulo, accione un motorreductor a través de un puente H L298N, generando una respuesta proporcional a la inclinación detectada. Se busca aplicar fusión de sensores (filtro complementario), control con zona muerta y rampa de velocidad, así como mecanismos de seguridad ante fallas de comunicación I2C.

Descripción de la práctica

El sistema utiliza un Arduino UNO R4 WiFi que se comunica por el bus I2C (pines SDA=A4, SCL=A5) con un sensor MPU-6050 en la dirección 0x68 (pin AD0 conectado a GND). A partir de las lecturas del acelerómetro y el giroscopio se calcula el ángulo de inclinación (pitch) mediante un filtro complementario, combinando la estabilidad a largo plazo del acelerómetro con la respuesta rápida del giroscopio.

El ángulo calculado se traduce en una señal PWM que controla un motorreductor a través de un módulo puente H L298N: el pin ENA (D9) regula la velocidad mediante PWM, mientras que IN1 (D8) e IN2 (D7) determinan el sentido de giro. El sistema aplica una zona muerta de 5° (para evitar movimiento ante pequeñas vibraciones), un ángulo máximo de 45° (saturación de la señal) y una rampa de aceleración que evita cambios bruscos de velocidad.

Como medida de seguridad, el programa coloca las salidas del motor en bajo antes de configurar los pines como salida, y recomienda una resistencia de 10 kΩ entre ENA y GND para mantener el motor deshabilitado mientras el Arduino arranca o se reinicia. Si el sensor deja de responder o el intervalo de control se alarga demasiado, el sistema entra en un estado de paro de seguridad y detiene el motor, intentando reconectar el sensor periódicamente hasta recuperarse.

Adicionalmente, el sistema utiliza la matriz de LED integrada del Arduino UNO R4 WiFi para mostrar de forma visual el estado del sistema: un marco completo cuando el sensor está centrado, una barra que se desplaza según la inclinación, o una 'X' cuando el sistema está detenido o en falla.

Material utilizado

- Arduino UNO R4 WiFi.
- Sensor acelerómetro/giroscopio MPU-6050.
- Módulo puente H L298N.
- Motorreductor (DC con reductor).
- Resistencia de 10 kΩ (entre ENA y GND, para mantener el motor deshabilitado durante el arranque).
- Protoboard y cables de conexión.
- Fuente de alimentación externa para el motor, acorde a sus especificaciones.

Conexiones

Motorreductor — Puente H L298N

Señal	Pin Arduino	Función
ENA	D9	Control de velocidad por PWM (sin jumper ENA en el módulo)
IN1	D8	Control de sentido de giro
IN2	D7	Control de sentido de giro

Sensor MPU-6050

Señal	Pin Arduino	Función
SDA	A4	Datos del bus I2C
SCL	A5	Reloj del bus I2C
AD0	GND	Fija la dirección I2C en 0x68

Nota de seguridad: se debe colocar una resistencia de 10 k Ω entre ENA y GND para mantener el motor deshabilitado mientras el Arduino arranca o se reinicia, evitando movimientos no deseados durante ese periodo.

Funcionamiento del programa

Máquina de estados

El sistema opera mediante cuatro estados: BLOQUEADO (fallo inicial del sensor, el motor nunca se habilita), ACTIVO (control normal en funcionamiento), PARO (detención de seguridad tras una falla) y RECUPERANDO (reintentando restablecer la comunicación con el sensor antes de volver a ACTIVO).

Cálculo del ángulo (filtro complementario)

El ángulo de inclinación se obtiene combinando el ángulo calculado a partir del acelerómetro (estable a largo plazo, pero sensible a vibraciones) con la integración de la velocidad angular del giroscopio (respuesta rápida, pero con deriva a largo plazo). El filtro complementario pondera ambas fuentes según una constante de tiempo (TAU_FILTRO), dando como resultado una estimación de ángulo estable y de respuesta rápida.

Control del motor

A partir del ángulo filtrado se calcula un valor PWM objetivo: si la inclinación es menor a la zona muerta (5°) el motor permanece apagado; si supera el ángulo máximo (45°) la señal se satura al PWM máximo. Entre ambos límites, el PWM crece de forma proporcional entre un mínimo (90) y un máximo (255). El PWM aplicado no salta directamente al valor objetivo, sino que se ajusta gradualmente mediante una

rampa (PASO_RAMPA), evitando cambios bruscos de velocidad y, al invertir el sentido de giro, pasando primero por cero.

Seguridad ante fallas

Si la lectura del sensor falla, o si el intervalo entre lecturas se vuelve demasiado largo (mayor a 100 ms), el sistema detiene el motor de inmediato y entra en estado de PARO, informando el motivo por el Monitor Serie. Periódicamente, el programa intenta reiniciar el bus I2C y verificar si el sensor volvió a responder; de ser así, pasa a un estado de RECUPERANDO antes de reanudar el control normal, evitando reintegrar el tiempo en que el sistema estuvo detenido.

Indicador visual (matriz de LED)

La matriz de LED integrada del Arduino UNO R4 WiFi se actualiza periódicamente para reflejar el estado del sistema: muestra una 'X' cuando el sistema está detenido o en falla inicial, un marco completo cuando la inclinación está dentro de la zona central ($\pm 2^\circ$), o una barra horizontal que se desplaza verticalmente según el grado de inclinación detectado.

Código

A continuación se presenta el código completo utilizado en la práctica.

```
/*
  Practica 3.2.3: Inclinometro con control de motorreductor.
  Arduino UNO R4 WiFi + MPU-6050 + modulo puente H L298N.

  Motor: ENA=D9 (sin jumper ENA), IN1=D8, IN2=D7.
  Sensor: bus Wire, SDA=A4, SCL=A5; AD0 a GND (direccion 0x68).
  Consultar Guia Inclinometro R4.md antes de conectar la alimentacion.
  Poner una resistencia de 10 kohm entre ENA y GND: mantiene el motor
  deshabilitado mientras el Arduino arranca o se reinicia.

  Mantener el sensor QUIETO durante las 500 muestras de calibracion.
  No utiliza WiFi ni necesita una biblioteca externa para el MPU-6050.
*/

#include <Arduino.h>
#include <Wire.h>
#include <Arduino_LED_Matrix.h>
#include <math.h>
#include <string.h>

const uint8_t PIN_ENA = 9;
const uint8_t PIN_IN1 = 8;
const uint8_t PIN_IN2 = 7;
const uint8_t DIRECCION_MPU = 0x68;

const float ZONA_MUERTA = 5.0f;
const float ZONA_CENTRO = 2.0f;
const float ANGULO_MAXIMO = 45.0f;
const int PWM_MINIMO = 90;
const int PWM_MAXIMO = 255;
const int PASO_RAMPA = 9; // 29 pasos x 20 ms = 580 ms.
const float TAU_FILTRO = 0.50f; // Alfa aproximado: 0.98 a 100 Hz.
const float SENTIDO_PITCH = 1.0f; // Cambiar a -1.0f si queda invertido.

const uint32_t PERIODO_SENSOR = 10;
const uint32_t PERIODO_MOTOR = 20;
const uint32_t PERIODO_SERIE = 500;
```

```

const uint32_t PERIODO_MATRIZ = 50;
const uint32_t PERIODO_REINTENTO = 250;
const uint32_t ESPERA_DESPERTAR = 100;
const unsigned int TIMEOUT_I2C_US = 5000;
const uint16_t MUESTRAS_CALIBRACION = 500;

struct LecturaIMU {
    float ax, ay, az;                // Aceleracion en g.
    float gy;                        // Giro sobre Y, en grados/segundo.
};

enum EstadoSistema { BLOQUEADO, ACTIVO, PARO, RECUPERANDO };

// Prototipos explicitos: tambien permiten usar LecturaIMU en el IDE Arduino.
bool escribirRegistro(uint8_t registro, uint8_t valor);
bool leerRegistros(uint8_t registro, uint8_t *datos, size_t cantidad);
bool identificarIMU();
bool configurarIMU();
bool leerIMU(LecturaIMU &lectura);
float pitchAcelerometro(const LecturaIMU &lectura);
bool calibrarGiroscopio();
void iniciarBus();
void detenerMotor();
void aplicarMotor();
int calcularObjetivo(float angulo);
void actualizarRampa();
void activarParo(const char *motivo);
void actualizarSensor();
void intentarRecuperacion(uint32_t ahora);
void informarCambios();
void actualizarMatriz();

ArduinoLEDMatrix matriz;
EstadoSistema estado = BLOQUEADO;
float offsetGy = 0.0f;
float pitch = 0.0f;
int pwmObjetivo = 0;                // El signo indica la direccion.
int pwmActual = 0;                  // PWM aplicado, NO velocidad medida.
uint32_t ultimoSensor = 0, ultimoMotor = 0, ultimoSerie = 0;
uint32_t ultimaMatriz = 0, ultimoReintento = 0, inicioRecuperacion = 0;
uint32_t ultimaLecturaUs = 0;
bool primerReporte = true;
int ultimoGrado = 0, ultimoPorcentaje = 0, ultimoSentidoMotor = 0;
int ultimoCentro = -1, ultimaIntensidad = -1, ultimoSentidoSensor = 0;

void iniciarBus() {
    Wire.begin();
    Wire.setClock(100000);           // I2C a 100 kHz.
    Wire.setWireTimeout(TIMEOUT_I2C_US); // API del paquete UNO R4 / Renesas.
}

bool escribirRegistro(uint8_t registro, uint8_t valor) {
    Wire.beginTransaction(DIRECCION_MPU);
    Wire.write(registro);
    Wire.write(valor);
    return Wire.endTransmission(true) == 0;
}

bool leerRegistros(uint8_t registro, uint8_t *datos, size_t cantidad) {
    Wire.beginTransaction(DIRECCION_MPU);
    Wire.write(registro);
    if (Wire.endTransmission(false) != 0) return false;
    if (Wire.requestFrom(DIRECCION_MPU, cantidad, true) != cantidad) return false;
    for (size_t i = 0; i < cantidad; ++i) {
        if (!Wire.available()) return false;
        datos[i] = (uint8_t)Wire.read();
    }
}

```

```

    return true;
}

bool identificarIMU() {
    uint8_t identificacion = 0;
    return leerRegistros(0x75, &identificacion, 1) && identificacion == 0x68;
}

bool configurarIMU() {
    return escribirRegistro(0x6B, 0x01) && // Despertar, reloj PLL giroscopio X.
        escribirRegistro(0x6C, 0x00) && // Habilitar acelerometro y giroscopio.
        escribirRegistro(0x1B, 0x00) && // Giroscopio: +/-250 grados/s.
        escribirRegistro(0x1C, 0x00) && // Acelerometro: +/-2 g.
        escribirRegistro(0x1A, 0x03) && // Filtro interno: aproximadamente 44 Hz.
        escribirRegistro(0x19, 0x09); // 1000/(1+9) = 100 muestras/s.
}

bool leerIMU(LecturaIMU &lectura) {
    uint8_t datos[14];
    if (!leerRegistros(0x3B, datos, sizeof(datos))) return false;
    const int16_t ax = (int16_t)((uint16_t)datos[0] << 8 | datos[1]);
    const int16_t ay = (int16_t)((uint16_t)datos[2] << 8 | datos[3]);
    const int16_t az = (int16_t)((uint16_t)datos[4] << 8 | datos[5]);
    const int16_t gy = (int16_t)((uint16_t)datos[10] << 8 | datos[11]);
    lectura.ax = ax / 16384.0f;
    lectura.ay = ay / 16384.0f;
    lectura.az = az / 16384.0f;
    lectura.gy = gy / 131.0f;
    // Rechazar un vector nulo: no permite conocer la inclinacion.
    const float norma2 = lectura.ax * lectura.ax + lectura.ay * lectura.ay +
        lectura.az * lectura.az;
    return norma2 > 0.000001f;
}

float pitchAcelerometro(const LecturaIMU &lectura) {
    const float vertical = sqrtf(lectura.ay * lectura.ay + lectura.az * lectura.az);
    return SENTIDO PITCH * atan2f(-lectura.ax, vertical) * 180.0f / PI;
}

bool calibrarGiroscopio() {
    float suma = 0.0f;
    LecturaIMU lectura;
    for (uint16_t i = 0; i < MUESTRAS_CALIBRACION; ++i) {
        if (!leerIMU(lectura)) return false;
        suma += lectura.gy;
        delay(PERODO SENSOR); // Solo en setup; motor apagado.
    }
    offsetGy = suma / MUESTRAS_CALIBRACION;
    return true;
}

void detenerMotor() {
    analogWrite(PIN_ENA, 0); // Quitar potencia primero.
    digitalWrite(PIN_IN1, LOW);
    digitalWrite(PIN_IN2, LOW);
    pwmActual = 0;
    pwmObjetivo = 0;
}

void aplicarMotor() {
    if (pwmActual == 0) {
        analogWrite(PIN_ENA, 0);
        digitalWrite(PIN_IN1, LOW);
        digitalWrite(PIN_IN2, LOW);
        return;
    }
    digitalWrite(PIN_IN1, pwmActual > 0 ? HIGH : LOW);
}

```

```

digitalWrite(PIN_IN2, pwmActual > 0 ? LOW : HIGH);
analogWrite(PIN_ENA, abs(pwmActual));
}

int calcularObjetivo(float angulo) {
    const float magnitud = fabsf(angulo);
    if (magnitud < ZONA_MUERTA) return 0;
    const float limitado = magnitud > ANGULO_MAXIMO ? ANGULO_MAXIMO : magnitud;
    const float proporcional = (limitado - ZONA_MUERTA) /
                                (ANGULO_MAXIMO - ZONA_MUERTA);
    const int pwm = (int)(PWM_MINIMO + proporcional * (PWM_MAXIMO - PWM_MINIMO) + 0.5f);
    return angulo >= 0.0f ? pwm : -pwm;
}

void actualizarRampa() {
    if (estado != ACTIVO) {
        detenerMotor();
        return;
    }
    // Al invertir: primero llegar a cero. La direccion cambia en otro paso.
    int destino = pwmObjetivo;
    if ((pwmActual > 0 && destino < 0) || (pwmActual < 0 && destino > 0)) destino = 0;
    if (pwmActual < destino) {
        pwmActual += PASO_RAMPA;
        if (pwmActual > destino) pwmActual = destino;
    } else if (pwmActual > destino) {
        pwmActual -= PASO_RAMPA;
        if (pwmActual < destino) pwmActual = destino;
    }
    aplicarMotor();
}

void activarParo(const char *motivo) {
    detenerMotor(); // Sin rampa en una falla.
    if (estado == ACTIVO) {
        Serial.print("PARO DE SEGURIDAD: ");
        Serial.println(motivo);
    }
    estado = PARO;
    ultimoReintento = millis();
}

void actualizarSensor() {
    LecturaIMU lectura;
    if (!leerIMU(lectura)) {
        activarParo("fallo de lectura del MPU-6050");
        return;
    }
    const uint32 t ahoraUs = micros();
    const float dt = (uint32_t)(ahoraUs - ultimaLecturaUs) / 1000000.0f;
    ultimaLecturaUs = ahoraUs;
    if (dt <= 0.0f || dt > 0.10f) {
        activarParo("intervalo de control demasiado largo");
        return;
    }
    const float anguloAccel = pitchAcelerometro(lectura);
    const float velocidadGy = SENTIDO_PITCH * (lectura.gy - offsetGy);
    const float alfa = TAU_FILTRO / (TAU_FILTRO + dt);
    pitch = alfa * (pitch + velocidadGy * dt) + (1.0f - alfa) * anguloAccel;
    pwmObjetivo = calcularObjetivo(pitch);
}

void intentarRecuperacion(uint32 t ahora) {
    if (estado == PARO && (uint32_t)(ahora - ultimoReintento) >= PERIODO_REINTENTO) {
        ultimoReintento = ahora;
        Wire.end();
        iniciarBus();
        if (identificarIMU() && configurarIMU()) {

```

```

        estado = RECUPERANDO;
        inicioRecuperacion = millis();
    }
} else if (estado == RECUPERANDO &&
    (uint32_t)(ahora - inicioRecuperacion) >= ESPERA_DESPERTAR) {
    LecturaIMU lectura;
    if (!leerIMU(lectura)) {
        activarParo("sensor aun no disponible");
        return;
    }
    // Conservar la calibracion; reiniciar angulo y rampa, sin integrar la pausa.
    pitch = pitchAcelerometro(lectura);
    detenerMotor();
    ultimaLecturaUs = micros();
    ultimoSensor = ultimoMotor = millis();
    primerReporte = true;
    ultimoCentro = -1;
    estado = ACTIVO;
    Serial.println("SENSOR RECUPERADO: control habilitado nuevamente.");
}
}

void informarCambios() {
    if (estado != ACTIVO || !Serial) return;
    const float magnitud = fabsf(pitch);
    const int centro = magnitud <= ZONA_CENTRO ? 1 : 0;
    const int grado = (int)roundf(pitch);
    const int porcentaje = (abs(pwmActual) * 100 + PWM_MAXIMO / 2) / PWM_MAXIMO;
    const int sentidoMotor = (pwmActual > 0) - (pwmActual < 0);
    const int sentidoSensor = centro ? 0 : (pitch > 0.0f ? 1 : -1);
    const int intensidad = magnitud < 15.0f ? 0 : (magnitud < 30.0f ? 1 : 2);
    if (!primerReporte && grado == ultimoGrado && porcentaje == ultimoPorcentaje &&
        sentidoMotor == ultimoSentidoMotor && centro == ultimoCentro &&
        intensidad == ultimaIntensidad && sentidoSensor == ultimoSentidoSensor) return;

    if (centro != ultimoCentro) {
        if (centro) Serial.println("ENTRA AL CENTRO: sensor dentro de +/-2 grados.");
        else if (ultimoCentro == 1) Serial.println("SALE DEL CENTRO.");
    }
    Serial.print("Inclinacion: ");
    Serial.print(centro ? "centrado" : (sentidoSensor > 0 ? "adelante" : "atras"));
    Serial.print(" | ");
    Serial.print(grado);
    Serial.print(" grados | ");
    Serial.print(intensidad == 0 ? "leve" : (intensidad == 1 ? "moderada" : "fuerte"));
    Serial.print(" | Motor: ");
    Serial.print(sentidoMotor == 0 ? "detenido" : (sentidoMotor > 0 ? "adelante" : "reversa"));
    Serial.print(" | PWM: ");
    Serial.print(porcentaje);
    Serial.println(" %");

    ultimoGrado = grado;
    ultimoPorcentaje = porcentaje;
    ultimoSentidoMotor = sentidoMotor;
    ultimoCentro = centro;
    ultimaIntensidad = intensidad;
    ultimoSentidoSensor = sentidoSensor;
    primerReporte = false;
}

void actualizarMatriz() {
    uint8_t imagen[8][12] = {};
    if (estado != ACTIVO) {
        for (int fila = 0; fila < 8; ++fila) { // X: paro o fallo inicial.
            imagen[fila][fila + 2] = 1;
            imagen[fila][9 - fila] = 1;
        }
    } else if (fabsf(pitch) <= ZONA_CENTRO) {

```

```

        for (int fila = 0; fila < 8; ++fila)
            for (int columna = 0; columna < 12; ++columna)
                if (fila == 0 || fila == 7 || columna == 0 || columna == 11)
                    imagen[fila][columna] = 1;
    } else {
        const float limitado = constrain(pitch, -ANGULO_MAXIMO, ANGULO_MAXIMO);
        const int fila = (int)(3.0f - limitado * 3.0f / ANGULO_MAXIMO + 0.5f);
        imagen[fila][5] = imagen[fila][6] = 1;
        imagen[fila + 1][5] = imagen[fila + 1][6] = 1;
    }
    matriz.renderBitmap(imagen, 8, 12);
}

void setup() {
    // PRIMERA ACCION: fijar las salidas de motor a nivel bajo.
    digitalWrite(PIN_ENA, LOW);
    digitalWrite(PIN_IN1, LOW);
    digitalWrite(PIN_IN2, LOW);
    pinMode(PIN_ENA, OUTPUT);
    pinMode(PIN_IN1, OUTPUT);
    pinMode(PIN_IN2, OUTPUT);
    analogWriteResolution(8);
    detenerMotor();

    Serial.begin(115200);
    const uint32_t inicioSerie = millis();
    while (!Serial && (uint32_t)(millis() - inicioSerie) < 1500) delay(1);
    matriz.begin();
    iniciarBus();

    if (!identificarIMU() || !configurarIMU()) {
        Serial.println("ERROR INICIAL: MPU-6050 no responde 0x68 o no se configura.");
        Serial.println("Motor apagado. Revisar conexiones y pulsar RESET.");
        actualizarMatriz();
        return; // BLOQUEADO: no arranca con sensor malo.
    }
    delay(ESPERA_DESPERTAR);
    Serial.println("MPU-6050 detectado. Mantener sensor QUIETO durante 5 segundos.");
    if (!calibrarGiroscopio()) {
        Serial.println("ERROR INICIAL: fallo durante la calibracion. Pulsar RESET.");
        actualizarMatriz();
        return;
    }
    LecturaIMU lectura;
    if (!leerIMU(lectura)) {
        Serial.println("ERROR INICIAL: no se pudo obtener el angulo inicial.");
        actualizarMatriz();
        return;
    }
    pitch = pitchAcelerometro(lectura);
    ultimaLecturaUs = micros();
    ultimoSensor = ultimoMotor = ultimoSerie = ultimaMatriz = millis();
    estado = ACTIVO;
    Serial.print("Calibracion lista. Offset Y: ");
    Serial.print(offsetGy, 4);
    Serial.println(" grados/s. Sistema listo.");
    actualizarMatriz();
}

void loop() {
    const uint32_t ahora = millis();
    if (estado == ACTIVO && (uint32_t)(ahora - ultimoSensor) >= PERIODO_SENSOR) {
        ultimoSensor = ahora;
        actualizarSensor();
    }
    if (estado == PARO || estado == RECUPERANDO) intentarRecuperacion(millis());
    const uint32_t despues = millis();
    if ((uint32_t)(despues - ultimoMotor) >= PERIODO_MOTOR) {

```

```

    ultimoMotor = despues;
    actualizarRampa();
}
if ((uint32_t)(despues - ultimoSerie) >= PERIODO_SERIE) {
    ultimoSerie = despues;
    informarCambios();
}
if ((uint32_t)(despues - ultimaMatriz) >= PERIODO_MATRIZ) {
    ultimaMatriz = despues;
    actualizarMatriz();
}
// Sin delay(): las cuatro tareas se programan con millis()/micros().
}

```

Resultados

Durante las pruebas, el sistema identificó correctamente al sensor MPU-6050 en la dirección 0x68 y completó la calibración del giroscopio manteniendo el sensor quieto durante las 500 muestras iniciales, obteniendo un offset que compensó la deriva propia del giroscopio.

Al inclinar el sensor por encima de la zona muerta de 5°, el motor comenzó a girar en el sentido correspondiente, incrementando su velocidad de forma proporcional conforme aumentaba el ángulo, hasta saturarse en el PWM máximo al alcanzar los 45°. Al regresar el sensor a la posición central (dentro de $\pm 2^\circ$), el motor se detuvo y el Monitor Serie reportó el evento de entrada al centro.

La rampa de aceleración evitó cambios bruscos de velocidad y, en los cambios de sentido, el PWM aplicado pasó primero por cero antes de invertir la dirección del motor, protegiendo tanto al motor como al puente H. Al simular una desconexión del sensor, el sistema detuvo el motor de inmediato y entró en estado de paro, recuperando el control automáticamente una vez restablecida la comunicación I2C, sin necesidad de reiniciar manualmente el Arduino.

Conclusiones

1. El filtro complementario permitió obtener una estimación de ángulo más estable y confiable que utilizar únicamente el acelerómetro o el giroscopio por separado, combinando las ventajas de ambos sensores.
2. La zona muerta evitó que el motor respondiera a pequeñas vibraciones o ruido en la lectura, mientras que la rampa de aceleración protegió al sistema mecánico de cambios bruscos de velocidad y dirección.
3. Colocar las salidas de control del motor en nivel bajo desde el inicio del programa, junto con una resistencia de 10 k Ω entre ENA y GND, resultó fundamental para evitar movimientos no deseados durante el arranque o reinicio del Arduino.
4. La implementación de una máquina de estados (BLOQUEADO, ACTIVO, PARO, RECUPERANDO) permitió manejar de forma segura las fallas de comunicación con el sensor, evitando que el motor operara con datos inválidos o desactualizados.
5. El uso de millis() y micros() en lugar de delay() permitió que el sistema atendiera de forma simultánea la lectura del sensor, el control del motor, el reporte por Monitor Serie y la actualización de la matriz de LED, cada uno con su propio periodo de actualización.

6. La matriz de LED integrada del Arduino UNO R4 WiFi resultó útil como indicador visual inmediato del estado del sistema, complementando la información mostrada por el Monitor Serie.